

Чорновик до залікової контрольної роботи

Робоча чернетка. Деякі місця залишені як виправлення, щоб було видно хід думки, а не чистовий звіт.

Теоретична частина

1. Перевести двійкове число 1000 1011 1110 0011 у десяткову систему:

Спочатку групую по 4 біти:

$$1000\ 1011\ 1110\ 0011_2 = 8\text{ВЕЗ}_{16}.$$

Далі:

$$8\text{ВЕЗ}_{16} = 8 \cdot 16^3 + 11 \cdot 16^2 + 14 \cdot 16 + 3 = 35811_{10}.$$

Відповідь: 35811.

2. Перевести двійкове число 1000 1011 1110 0011 у шістнадцяткову систему:

$$1000\ 1011\ 1110\ 0011_2 = 8\text{ВЕЗ}_{16}.$$

3. За часовими діаграмами:

$$x = I\text{-НЕ}, y = \text{АБО-НЕ}.$$

Спочатку подумав, що y може бути АБО, але за діаграмою видно інверсію, тому $y = \text{АБО-НЕ}$.

4. Ознаки, що формуються в арифметико-логічних блоках під час арифметичних операцій:

Правильний варіант: $Z = 1$ визначається операцією АБО-НЕ над усіма розрядами результату; знак результату N дорівнює значенню старшого знакового розряду результату в модифікованому машинному коді $\text{ЗР}(n+2)$.

5. Ознака переповнення в знаковому розряді:

Вибрати:

- $\text{flag}\{\text{Overflow}\} = C_{\{n-1\}} \text{ XOR } C_n$, де $C_{\{n-1\}}$ - перенос у знаковий розряд, C_n - перенос із знакового розряду за межі розрядної сітки.
- Ознака переповнення встановлюється в 1, коли старший знаковий біт обох доданків однаковий, але знак результату відрізняється.

6. Навіщо в АЛП операції з дійсними числами виконуються над нормалізованими мантисами:

Для забезпечення однакового формату чисел і точності обчислень, а також для уникнення втрати значущих розрядів при додаванні або відніманні.

7. Нормалізована мантиса двійкового числа:

Вибрати:

- Це дробова частина числа з плаваючою комою, яка містить його значущі біти.

- Нормалізовані мантиси додатних чисел у розряді з вагою 2^{-1} завжди мають одиницю.

8. Чому неможливо виконувати алгебраїчне підсумовування і віднімання на звичайних суматорах у прямих машинних кодах:

Правильний варіант: у прямому коді знакові розряди і основні розряди мають оброблятися по-різному. Віднімання на звичайному суматорі виконують, якщо аргументи подані у зворотному або доповняному коді; тоді віднімання замінюється додаванням числа з протилежним знаком.

9. Додавання двійкових мантис зі знаками:

Дано:

$A = 0\ 10101$, $B = 0\ 0011010$.

Обидва числа додатні, тому додаємо мантиси:

```

  0.1010100
+ 0.0011010
-----
  0.1101110

```

Точний результат: $0\ 1101110$.

У 6-розрядній сітці без округлення: $0\ 11011$.

~~Можна було б округлити~~, але в умові просто записати в 6-розрядну сітку, тому беру обрізання.

10. Яка з мікросхем має пам'ять:

Лічильник.

11. Чим відрізняється однорозрядний напівсуматор від повного однорозрядного суматора:

Однорозрядний суматор за розрядним значенням x_i і y_i доданків і за значенням переносу z_i із сусіднього молодшого розряду формує значення розрядної суми S_i і переносу у наступний старший розряд P_i . Однорозрядний напівсуматор не враховує вхідний перенос z_i .

12. Восьмирозрядний суматор складається із:

Ланцюга із восьми повних однобітних суматорів.

13. Дешифратор / шифратор: правдиві твердження:

Вибрати:

- Неповний дешифратор має менше ніж 2^n виходів.
- Шифратор - це пристрій, що виконує функцію, зворотну дешифратору.
- Повний дешифратор має n входів і 2^n виходів.
- При подачі сигналу на один із входів шифратора, тобто унітарного коду, на виході повинен утворитися двійковий код.
- Дешифратор - це комбінаційна схема для перетворення двійкового позиційного коду на унітарний.

14. Мікрооперації, які можуть виконуватися на регістрах:

Вибрати:

- Зсув логічний.
- Зсув арифметичний.
- Зберігання.
- Запис.

- Встановлення в нуль, тобто очищення.

Функціональне моделювання в САПР ModelSim

15. Як можна реалізувати 16-розрядний регістр на ПЛІС (FPGA):

Знадобиться 16 логічних блоків ПЛІС, де кожен біт регістра реалізується за допомогою одного D-тригера (flip-flop).

16. З якою ціллю розробляється Testbench у САПР:

Для емуляції оточення розроблюваного пристрою і аналізу результатів.

17. Яку послідовність вхідних сигналів сформує Testbench:

Код:

```
@(posedge clk); out1=0; out2=0;
@(posedge clk); out1=0; out2=1;
@(posedge clk); out1=1; out2=0;
@(posedge clk); out1=1; out2=1;
```

Послідовність:

00 01 10 11.

18. Таблиця істинності у часовій області для неповного дешифратора 3-to-8 з входом EN:

Правильний варіант: 2.

Логіка така: при $EN = 0$ вихід $Y = 00000000$, при $EN = 1$ активується один вихід за адресою $A2 A1 A0$.

19. Скільки тактів буде тривати процес моделювання заданим Testbench:

Правильний варіант: вісім тактів.

Пояснення: 4 такти для комбінацій 00, 01, 10, 11, потім ще 1 такт окремо і repeat(3) додає ще 3 такти.

$$4 + 1 + 3 = 8.$$

20. Для тестування якого пристрою використовується Testbench з reg [1:0] sel, reg [3:0] d, wire y:

Правильний варіант: мультиплексор 4x1.

Пояснення: є 4 інформаційні входи d[3:0], 2-бітний вибір sel і один вихід y.

Практична частина

21. Адресний селектор зовнішнього пристрою:

Правильний варіант: PC = 29A4h.

Пояснення: інверсні входи селектора мають отримати 0, прямі входи - 1; A0 не впливає на вибір, за умовою беремо A0 = 0.

22. Практичне завдання за номером студентського квитка:

Номер: 4106.

Переведення у двійковий код:

4106_10 = 1000000001010_2

Далі вирівнюю число праворуч у 16-розрядній сітці, тобто дописую нулі ліворуч:

4106_10 = 0001 0000 0000 1010_2

Два 8-розрядні аргументи:

R1 = 00010000_2 = +16

R2 = 00001010_2 = +10

Обидва аргументи додатні, у доповняльному коді запис не змінюється:

A = 00010000

B = 00001010

Виконується додавання:

```
  00010000
+ 00001010
-----
  00011010 = 26
```

У 9-розрядному регістрі:

S = 000011010

Переповнення немає, бо A[7] = 0, B[7] = 0, S[7] = 0.

Далі виконується гілка без переповнення:

S = S >>> 2;
RESULT = S[8:7];

Після арифметичного зсуву праворуч на 2:

до зсуву: S = 000011010 = 26

після >>> 1: S = 000001101 = 13

S = 000000110 = 6

RESULT = S[8:7] = 00

Цифрова діаграма станів регістрів:

Етап	Операція	A	B	S	OVERFLOW	RESULT
0	Початкові аргументи	00010000	00001010	—	—	—
1	S = A + B	00010000	00001010	000011010	—	—
2	Перевірка overflow	00010000	00001010	000011010	0	—
3	S = S >> 2	00010000	00001010	000000110	0	—
4	RESULT = S[8:7]	00010000	00001010	000000110	0	00

Регістр C у цьому варіанті не змінюється, бо гілка з C = S виконується тільки при переповненні.

Ознаки для S:

S = 000000110

zero_flag = 0, бо S не дорівнює нулю

overflow_flag = 0, бо переповнення не було

sign_flag = 0, бо знаковий біт дорівнює 0

Результат у прямому коді: $00000110_2 = +6$.

Перевірка десятковими еквівалентами:

$$R1 = +16$$

$$R2 = +10$$

$$S = 16 + 10 = 26$$

$$S \ggg 2 = 26 / 4 = 6$$

У Google Forms для RESULT: 00.

23. Сторінкова пам'ять:

Умова: загальний об'єм пам'яті 516K x 16, об'єм однієї сторінки пам'яті 128K x 16.

Спочатку визначаю шину даних. Другий множник у записі пам'яті означає розрядність одного слова:

$$n = 16$$

Тобто ШД = 16 біт.

Одна сторінка має об'єм 128K x 16. Кількість адресованих слів на сторінці:

$$128K = 128 * 1024 = 2^7 * 2^{10} = 2^{17}$$

$$m2 = 17$$

Отже молодші біти адреси A16..A0 задають адресу всередині сторінки.

Загальний об'єм пам'яті:

$$516K = 516 * 1024 = 528384$$

$$2^{19} = 524288 < 528384 < 2^{20}$$

Тому для повної адресації потрібно 20 адресних бітів:

$$m = 20$$

Тобто ША = 20 біт.

Кількість сторінок:

$$N = \text{ceil}(516K / 128K)$$

$$N = \text{ceil}(4.03125)$$

$$N = 5 \text{ сторінок}$$

Тут саме 5 сторінок, бо 4 сторінки дають тільки:

$$4 * 128K = 512K$$

а потрібно 516K. ~~Спочатку можна було б взяти 4 сторінки~~, але це покриває тільки 512K, тому потрібна ще частина п'ятої сторінки.

Для вибору 5 сторінок треба дешифратор із трьома входами:

$$m1 = \text{ceil}(\log_2(5)) = 3$$

Старші адресні біти для вибору сторінки: A19, A18, A17.

Підсумкова таблиця параметрів:

Параметр	Значення
Загальний об'єм пам'яті	516K x 16
Об'єм однієї сторінки	128K x 16
Розрядність шини даних, ШД	16
Розрядність шини адреси, ША	20
Кількість сторінок N	5
Кількість входів дешифратора m_1	3
Розрядність адреси сторінки m_2	17

Таблиця істинності дешифратора 3 -> 8 для 5 сторінок:

A19	A18	A17	CS0	CS1	CS2	CS3	CS4	Сторінка
0	0	0	1	0	0	0	0	0
0	0	1	0	1	0	0	0	1
0	1	0	0	0	1	0	0	2
0	1	1	0	0	0	1	0	3
1	0	0	0	0	0	0	1	4
1	0	1	0	0	0	0	0	не використ.
1	1	0	0	0	0	0	0	не використ.
1	1	1	0	0	0	0	0	не використ.

Логічні рівняння:

$$CS0 = \sim A19 \& \sim A18 \& \sim A17$$

$$CS1 = \sim A19 \& \sim A18 \& A17$$

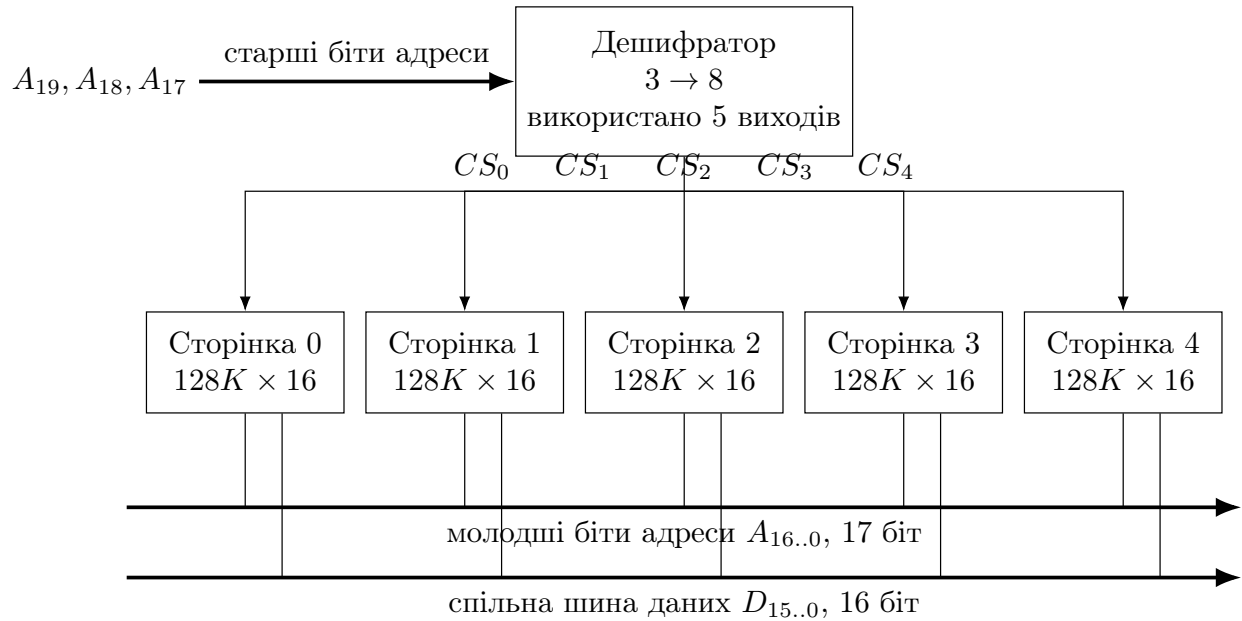
$$CS2 = \sim A19 \& A18 \& \sim A17$$

$$CS3 = \sim A19 \& A18 \& A17$$

$$CS4 = A19 \& \sim A18 \& \sim A17$$

Структурна схема багатосторінкової пам'яті:

Старші біти вибирають сторінку, молодші біти адресують слово всередині сторінки



Пояснення підключення: старші біти адреси $A_{19}..A_{17}$ подаються на дешифратор вибору сторінки, молодші $A_{16}..A_0$ йдуть паралельно на всі сторінки пам'яті, а спільна шина даних $D_{15}..D_0$ підключена до виходів сторінок. Активний сигнал CS_i дозволяє роботу тільки однієї сторінки. Комбінації 101, 110, 111 не використовуються.

У Google Forms:

Кількість сторінок: 5

Розрядність даних ШД: 16

Розрядність адреси ША: 20

Кількість входів дешифратора m1: 3

Розрядність адреси сторінки m2: 17